



47

MANUAL DE EXPRESS.

[DOCUMENTACIÓN OFICIAL.](#)

Introducción a Express.js	3
1. Instalar Express con npm.	6
2. Crear un servidor.	10
3. Añadir más rutas	16
Ejercicio.	17
Ejercicio: Mi primer servidor Express con frontend conectado	17
♦ Ruta raíz /	18
♦ Ruta /info	18
♦ Ruta /contacto	18
♦ Ruta /tienda	19
♦ Ruta no encontrada (404)	19
Recordatorios y pistas	20
4. Middlewares + API REST básica (CRUD)	20
Características principales	21
Tipos de middleware	22
Ejercicios.	25
Añade una ruta que devuelva HTML	27
5. Introducción al Modelo MVC (Modelo–Vista–Controlador)	28
6. API REST con Express (CRUD básico)	31

Estructura.	31
Modelo.	36
Controlador.	41
Axios. En el Frontend	44



Introducción a Express.js

“El siguiente paso después de Node”



¿Qué es Express?

Express.js (o simplemente Express) es un framework para Node.js que sirve para crear servidores web y APIs de forma más rápida, limpia y organizada.

Piensa que Node es como el motor de un coche: potente, pero un poco incómodo de usar si lo manejas pieza a pieza. Express sería el coche completo: ya tiene volante, pedales y salpicadero listos para conducirlo.



¿Por qué usar Express si ya sabemos Node?

Porque con Node puro ya vimos que:

- Crear un servidor con `http.createServer()` funciona, pero se hace largo y difícil de mantener.
- Leer el `req.url` y analizar rutas con `if` es poco práctico.
- Gestionar los datos del `body` o los errores es confuso.



Express nos soluciona todo eso con pocas líneas y más claridad.



¿Qué nos aporta Express?

1) Rutas mucho más fáciles

En Node:

```
if (req.url === "/saludo" && req.method === "GET") {  
  res.end("Hola desde Node");  
}
```

En Express:

```
app.get("/saludo", (req, res) => {  
  res.send("Hola desde Express");  
});
```

🎯 ¡Una línea y listo!

② Middleware: funciones intermedias inteligentes

Un middleware es como un filtro o ayudante que se ejecuta antes o después de atender la petición.

Sirve para tareas repetidas como:

- Convertir el cuerpo (body) en objeto JSON
- Comprobar si el usuario está autenticado
- Registrar peticiones, validar datos, etc.

Ejemplo:

```
app.use(express.json());
```

👉 A partir de ahí, cualquier req.body ya es un objeto listo para usar.

③ Respuestas rápidas y claras

En Node:

```
res.writeHead(200, {"Content-Type": "text/plain"});  
res.end("Todo OK");
```

En Express:

```
res.send("Todo OK");
```

📦 También puedes enviar JSON directamente:

```
res.json({ mensaje: "Todo OK" });
```

4 Proyectos organizados

Express nos anima a dividir el servidor en carpetas:

/backend

```
|— servidor.js
|— routes/
|   |— tareas.js
|   |— usuarios.js
|— controllers/
|   |— tareasController.js
|— data/
|   |— tareas.json
```

Cada carpeta tiene su función, y el código es más limpio y fácil de mantener.

5 Un ecosistema enorme

Express es el framework más usado en el mundo backend con JavaScript.

Gracias a eso, puedes:

- Conectarlo fácilmente a bases de datos (MongoDB, MySQL, etc.)
 - Crear APIs REST que luego consumen tus apps de React, Vue o React
 - Añadir librerías de seguridad, validación, sesiones, etc.
-

Conclusión

Express no sustituye a Node, sino que trabaja sobre él.

Podemos decir que:

🧠 Node es el cerebro

🔧 Express es el esqueleto que organiza todo

🔗 Las rutas son las venas por donde viajan los datos

Gracias a Express, podremos construir APIs completas, proyectos conectados al frontend, y servidores listos para aplicaciones profesionales.

🧰 Próximo paso en el manual:

➡ Instalar Express con npm

➡ Crear nuestro primer servidor con `app.listen()`

➡ Entender cómo funcionan las rutas y los métodos HTTP

1. Instalar Express con npm.

Objetivo: Crear nuestro primer proyecto y servidor con Express

Ahora que ya dominamos Node.js puro, vamos a dar el salto a Express, un framework que nos permite crear servidores web y APIs REST de forma más rápida, limpia y profesional.

En esta primera lección:

- Aprenderás a instalar y configurar un proyecto con Express.
- Crearás tu primer servidor funcional que responde a una petición.
- Entenderás cómo funcionan los métodos básicos (`app.get()`, `app.listen()`).

Paso 1. Crear la carpeta del proyecto. Crea una carpeta vacía para tu nuevo proyecto Express:

Paso 2. Iniciar el proyecto con npm

Inicializamos el proyecto de Node (igual que antes):

`npm init -y`

Esto genera un archivo llamado `package.json`, donde se guardarán las dependencias del proyecto (como Express).

Este archivo `package.json` es el que modificaremos como hacíamos en node.

Paso 3. Instalar Express

Instalamos el paquete Express desde npm: `npm install express`

Esto descargará Express y lo añadirá como dependencia en tu `package.json`.

Como verás se ha generado otro archivo `lock.json`. Ese archivo es el “registro técnico”. npm lo usa para recordar las versiones exactas de las librerías.

Garantiza que si otro compañero instala tu proyecto, tendrá exactamente las mismas versiones.

No se edita nunca a mano.

Paso 4. Ajustes importantes en package.json

En este paso vamos a realizar algunos cambios esenciales en el archivo `package.json` para que nuestro proyecto con Express funcione correctamente y sea más cómodo de ejecutar durante el desarrollo.

1) Añadir "type": "module"

Por defecto, Node.js utiliza el sistema de módulos antiguo llamado CommonJS, que emplea las funciones `require()` y `module.exports`.

Sin embargo, nosotros vamos a trabajar con el sistema moderno de módulos ES6, que usa `import` y `export`.

Para indicarle a Node que debe interpretar el código con esta sintaxis, añadimos la siguiente línea dentro del objeto principal del archivo package.json:

```
"type": "module",
```

② Añadir los scripts de ejecución

A continuación, configuramos dos scripts que nos permitirán iniciar el servidor de forma sencilla desde la terminal:

- "start": ejecuta el servidor normalmente usando Node.
- "dev": ejecuta el servidor con nodemon, una herramienta que reinicia el servidor automáticamente cada vez que guardamos cambios en los archivos.

El bloque completo quedaría así:

```
"scripts": {  
  "start": "node servidor.js",  
  "dev": "nodemon servidor.js"  
}
```

De esta forma, podremos ejecutar nuestro servidor con los comandos:

- npm start → para ejecución normal.
- npm run dev → para modo desarrollo con recarga automática.

③ Instalar nodemon (modo desarrollo)



¿Qué es nodemon?

nodemon es una herramienta muy útil durante el desarrollo en Node.js.

Su función principal es vigilar los cambios que realizas en tus archivos del proyecto y reiniciar automáticamente el servidor cada vez que guardas.

👉 Gracias a ello, no necesitas detener manualmente el servidor ni volver a ejecutar `node servidor.js` cada vez que hagas una modificación. Esto agiliza muchísimo el trabajo mientras programas.

✅ Instalación (solo para desarrollo):

Para instalar nodemon como dependencia de desarrollo, escribe en la terminal:
`npm install -D nodemon`

📦 Nota:

El modificador `-D` (también puedes escribirlo como `--save-dev`) indica que esta herramienta solo se utilizará en el entorno de desarrollo, y no se incluirá cuando despliegues tu aplicación en producción pues consume recursos extras que no son necesarios e incluso podría romper tu despliegue. De esta manera, mantenemos nuestro proyecto más limpio y optimizado.

En desarrollo para inicializar nuestro proyecto usaremos
`npm run dev`

Ejemplo de package json con los cambios que hemos comentado. Recuerda que no admite comentarios.

```
{  
  "name": "express",  
  "version": "1.0.0",  
  "description": "",  
  "main": "index.js",  
  "type": "module",  
  "scripts": {
```

```
"start": "node servidor.js",  
  
"dev": "nodemon servidor.js"  
  
},  
  
"keywords": [],  
  
"author": "",  
  
"license": "ISC",  
  
"dependencies": {  
  
  "express": "^5.1.0"  
  
},  
  
"devDependencies": {  
  
  "nodemon": "^3.1.10"  
  
}  
  
}
```

2. Crear un servidor.

Hasta ahora, hemos visto que con Node.js puro podemos crear un servidor desde cero usando el módulo nativo http.

Esto nos permite controlar todos los detalles del proceso, pero también hace que el código sea más extenso y técnico.

Por ejemplo, en Node necesitamos escribir manualmente las cabeceras de respuesta, manejar el objeto req (petición) y res (respuesta), y comprobar con condicionales qué ruta ha solicitado el navegador.

Esto implica más líneas de código y menos legibilidad.

En cambio, Express.js se apoya en Node para ofrecer una forma más directa, limpia y declarativa de hacer exactamente lo mismo.

Con solo unas pocas líneas, podemos levantar un servidor, definir rutas y enviar respuestas sin preocuparnos por los detalles de bajo nivel.

La diferencia más clara es que con Express ya no usamos `http.createServer()`, sino que creamos una aplicación con `express()`.

A partir de ahí, podemos definir rutas con métodos como `app.get()` o `app.post()`, y responder con `res.send()` o `res.json()` de manera intuitiva.

Además, Express se encarga automáticamente de las cabeceras y del tipo de contenido, lo que simplifica enormemente el trabajo del desarrollador.

Podemos resumirlo así:

Con Node puro, programamos el motor pieza a pieza.

Con Express, conducimos un coche listo para funcionar, con todo el sistema preparado para atender rutas y peticiones sin esfuerzo.

Por tanto, Express no sustituye a Node.js, sino que lo utiliza internamente.

Podríamos decir que Node es la base, el motor, mientras que Express añade la estructura y las herramientas necesarias para crear aplicaciones web completas y escalables de forma profesional.

Esta simplificación no solo reduce el tiempo de desarrollo, sino que también mejora la claridad, la organización del código y la mantenibilidad del proyecto.

Por eso, en los siguientes apartados aprenderemos a aprovechar todas sus ventajas: rutas, middleware, respuestas personalizadas y creación de APIs REST.

[Ejemplo de servidor en Express y en Node.](#)

Ahora hazlo tú.

❶ Crear el archivo principal servidor.js

En la raíz de tu proyecto (donde está tu `package.json`), crea un archivo nuevo.

```
// servidor.js
import express from "express";

const app = express();
```

```
// Ruta principal
app.get("/", (req, res) => {
  res.send("🌍 ¡Hola mundo desde Express!");
});

// Iniciar servidor
app.listen(3000, () => {
  console.log("✅ Servidor escuchando en http://localhost:3000");
});
```

`const app = express();`

Crea una aplicación de Express. app es tu “servidor + router + middleware”.

app no es una palabra reservada si no de convención, Es la variable donde se crea la app express con todos los métodos que express trae.

```
// Ruta principal
app.get("/", (req, res) => {
  res.send("🌍 ¡Hola mundo desde Express!");
});
```

`app.get("/")`: define una ruta para peticiones GET a /.

`(req, res)`: objetos de petición y respuesta. En Express vienen “vitaminados”.

`res.send(...)`: envía respuesta y cierra la conexión (setea cabeceras apropiadas si hace falta).

¿Cómo era esto antes en node?

Así era el equivalente. Seguimos con los mismos conceptos de res y req pero cambia la sintaxis facilitandola.

```
import http from "node:http";

const server = http.createServer((req, res) => {
  // comprobamos la ruta y el método manualmente
  if (req.method === "GET" && req.url === "/") {
    res.writeHead(200, { "Content-Type": "text/plain" });
    res.end("🌈 ¡Hola mundo desde Node!");
  } else {
    res.writeHead(404, { "Content-Type": "text/plain" });
    res.end("❌ Página no encontrada");
  }
});

server.listen(3000, () => {
  console.log("Servidor Node escuchando en http://localhost:3000");
});
```



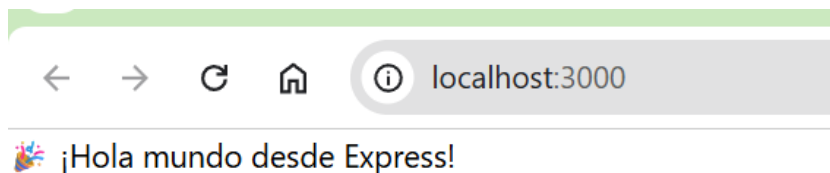
Aquí se pone a escuchar al puerto. Esto es igual que en Node.

// Iniciar servidor

```
app.listen(3000, () => {
  console.log("✅ Servidor escuchando en http://localhost:3000");
});
```

Comprobamos que todo funciona.

Ejecutas npm run dev



En definitiva, app no es el servidor HTTP, sino una función manejadora enriquecida por Express.

Express crea internamente un objeto res (de respuesta) con métodos más cómodos encima de los de Node. Internamente Express hace por ti:

1. Detecta el tipo de dato (string, object, Buffer, etc.).
2. Ajusta automáticamente el código de estado (200 por defecto).
3. Pone las cabeceras necesarias.

Estos son los métodos fundamentales que incluye res

<code>res.send()</code>	Envía texto, HTML o Buffer	<code>res.writeHead() + res.end()</code>
<code>res.json()</code>	Envía JSON y ajusta cabecera <code>application/json</code>	Manual con <code>JSON.stringify()</code>
<code>res.status(code)</code>	Cambia el código de estado	<code>res.writeHead(code)</code>
<code>res.sendFile(path)</code>	Envía un archivo	Leer con <code>fs</code> + <code>res.end()</code>
<code>res.redirect(url)</code>	Redirige al usuario	Manual con código 302 y <code>Location</code>

Ejemplo de res.json

// Ruta "/info"

```
app.get("/info", (req, res) => {  
  
  res.json({  
  
    nombre: "Luis Gómez",  
  
    curso: "Desarrollo Web con Express",  
  
    año: 2025  
  
  });  
  
});
```

De forma que si creamos un botón en html y le doy esta ruta /info desde nuestro servidor nos va a llegar ese objeto. Probamos y nos da un error ¿Qué falla? ¿Os acordáis de las cabeceras Cors? Pues evidentemente siguen haciendo falta pero ahora es mucho más sencillo.

Vamos a instalar cors en nuestro proyecto

```
npm i cors
```

Una vez instalado importamos

```
import cors from "cors";
```

y ahora ya llamamos al método cors antes de las rutas

```
app.use(cors()); // permitimos todos los orígenes y todos los métodos
```

Habilita CORS universal:

- Permite cualquier origen (*)
- Permite todos los métodos (GET, POST, PUT, DELETE, etc.)

Evidentemente si estamos en entorno real tendríamos que pensar en limitar las Cors

```
const corsOptions = {  
  origin: 'http://example.com', // solo permite peticiones desde example.com  
  methods: 'GET, POST, PUT, DELETE', // métodos permitidos  
  allowedHeaders: 'Content-Type'  
};  
app.use(cors(corsOptions));
```

Con esto acabamos de implementar nuestra primera middleware que no son más que módulos de terceros que amplían considerablemente las posibilidades de express ahorrándonos mucho código.

3. Añadir más rutas

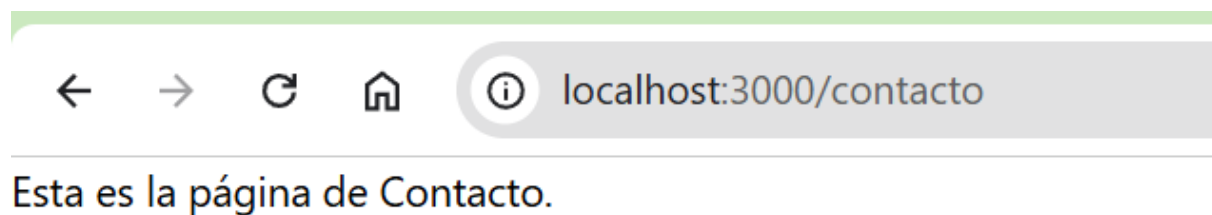
Express se basa en **rutas**.

Cada una responde a una **URL** concreta y un **método HTTP** (GET, POST, PUT, DELETE...).

Con express nos olvidamos de los if

```
///
app.get("/contacto", (req, res) => {
  res.send("Esta es la página de Contacto.");
});
```

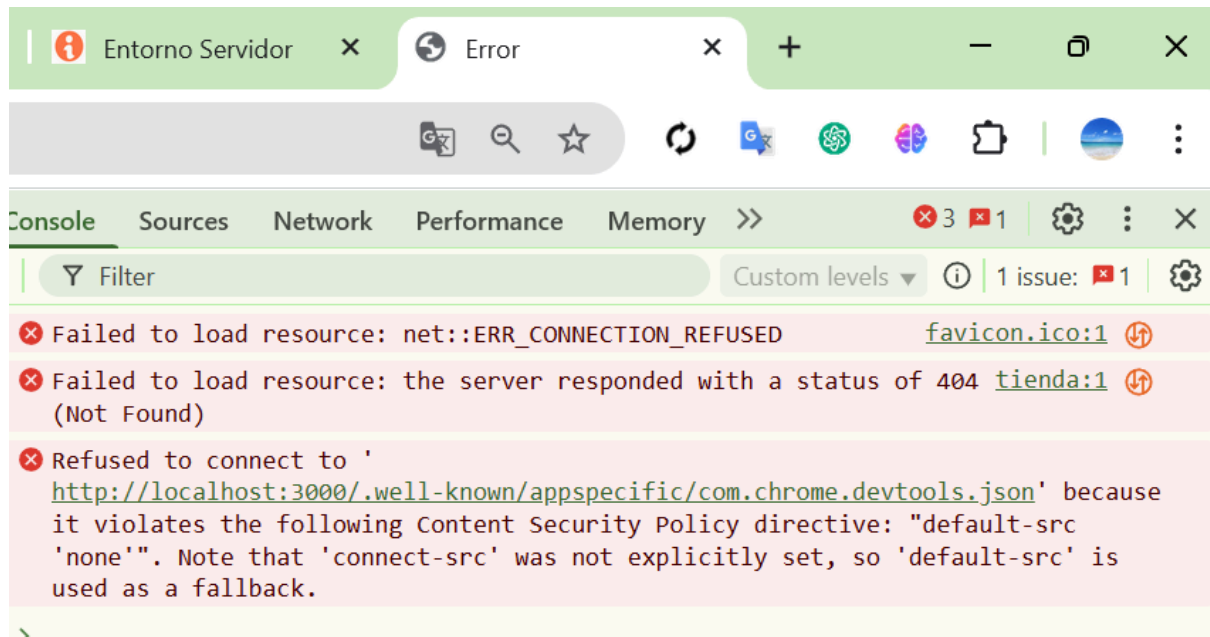
Ejecutas vas a la ruta y verás



Por último, nos acostumbramos a poner una ruta por defecto para aquellas rutas que no existen

```
app.use((req, res) => {
  res.status(404).send("404 - Página no encontrada");
});
```

Si el usuario pone una ruta que no existe nos va a lanzar un mensaje de ruta no encontrada pero ¿dónde lo lanza?



Ejercicio.

Crea un servidor con tres rutas la inicial, otra para tienda y una última para rutas inexistentes.

Aquí tienes la explicación. Consulta lo visto en fetch en node.

🧩 Ejercicio: Mi primer servidor Express con frontend conectado

Crear un servidor con **Express** que responda de forma diferente según la ruta a la que se acceda desde el navegador o el frontend (HTML + JavaScript).

Tu misión es crear un **servidor Express** que escuche en el puerto **3000** y tenga **estas rutas** con las siguientes respuestas:

♦ Ruta raíz /

👉 Devuelve un mensaje de texto sencillo.

Por ejemplo:

“Inicio 🏠” o “Bienvenido al servidor Express 🙌”

♦ Ruta /info

Devuelve un **objeto JSON** con información sobre el curso.

Debe incluir al menos:

- Tu nombre o el del autor.
- El nombre del curso (por ejemplo, “Desarrollo Web con Express”).
- El año actual (2025).

Ejemplo orientativo (solo para que veas la estructura):

```
{  
  "nombre": "Luis Gómez",  
  "curso": "Desarrollo Web con Express",  
  "año": 2025  
}
```

♦ Ruta /contacto

Devuelve **contenido HTML**, no JSON ni texto plano.

Debe incluir:

- Un título o encabezado.
- Un párrafo con un enlace de correo electrónico, tipo:
"Puedes contactarnos en `contacto@ejemplo.com`".

(Pista: pueden usar etiquetas HTML dentro de las comillas, como `<h3>` o `<a href="mailto:...">`.)

♦ Ruta `/tienda`

👉 Devuelve un mensaje de texto indicando que es la sección de tienda.

Por ejemplo:

"Bienvenido a la tienda 🎁"

♦ Ruta no encontrada (404)

Si el usuario escribe una dirección inexistente (por ejemplo `/abc`), el servidor debe devolver un mensaje HTML indicando **Error 404**.

Debe incluir:

- Un título como "❌ Error 404".
- Un texto explicando que la página no existe.

(Esta parte se hace al final con un `app.use()` que capture todas las rutas no definidas.)

Recordatorios y pistas

💡 1. No olvides importar y usar el middleware `cors()` para que el frontend pueda hacer peticiones.

💡 2. Cada ruta se crea con `app.get(ruta, (req, res) => { ... })`.

💡 3. Usa `res.send()` para enviar texto o HTML y `res.json()` para enviar JSON.

💡 4. Ejecuta el servidor con:

Construir una página web sencilla (**HTML + JavaScript**) que se comuniquen con el servidor Express creado en el ejercicio anterior.

La página debe enviar peticiones con `fetch()` a las distintas rutas del servidor y mostrar los resultados en pantalla.

Crea cuatro botones, cada uno con un identificador único (`id`) para poder seleccionarlos desde JavaScript.

4. Middlewares + API REST básica (CRUD)

Al finalizar esta unidad, serás capaz de saber:

- Qué es un middleware y por qué es el corazón de Express.
- Cómo ayudan los middlewares a ordenar, proteger y automatizar tareas dentro de un servidor.
- Cuáles son los middlewares esenciales que debe tener toda API profesional.
- Cómo estructurar rutas y respuestas en Express sin complicaciones.
- Cómo preparar una API REST básica (CRUD) para conectar con el frontend.

Un middleware es, literalmente, una función intermedia que se ejecuta entre la petición (`req`) que llega al servidor y la respuesta (`res`) que se envía al cliente.

Imagina que cada petición HTTP pasa por una cadena de pasos o filtros, y cada filtro puede:

- Registrar lo que pasa (logs),
- Validar los datos,
- Permitir o bloquear el acceso,
- Convertir el cuerpo de la petición (body) en JSON,
- Añadir cabeceras de seguridad,
- O incluso terminar la respuesta directamente.

En Node “puro”, tú montabas estos pasos a mano (leer body con `req.on('data')`, imprimir logs, poner cabeceras...). En Express, esos pasos se convierten en middlewares reutilizables.

Cada middleware puede inspeccionar, modificar o tomar decisiones sobre la petición o la respuesta, y luego decide si deja pasar la petición al siguiente middleware o la termina ahí.

De hecho nosotros ya hemos instalado antes una middleware: cors.

Características principales

- Cada función middleware recibe tres parámetros: `req` (la petición que hizo el cliente), `res` (la respuesta que se enviará al cliente), y `next` (una función que llama al siguiente middleware en la cadena).
- Puede ejecutar cualquier tarea: validar datos, modificar la solicitud o respuesta, autenticar usuarios, registrar logs, etc. Por ejemplo un usuario que hace login si la clave es correcta validaremos ese get y llamaremos a `next` para que lo pase a la página de perfil.
- Si el middleware termina la respuesta, no hace falta llamar a `next()`. Si solo realiza una parte del trabajo y desea que la petición siga su camino, debe llamar a `next()` para que la cadena continúe. Si olvida llamar a `next()`, la petición se queda “colgada”.

Tipos de middleware

- Middleware de aplicación: Se aplica a toda la app y todas las rutas.
- Middleware de ruta: Solo funciona en rutas específicas.
- Middleware de terceros: Funciones ya hechas, como body-parser para datos en JSON.
- Middleware de manejo de errores: Detectan y gestionan errores en la app

 Middleware (Express)	 ¿Para qué sirve?	 Qué hace exactamente	 Equivalente en Node puro
<code>express.json()</code>	Leer el cuerpo JSON de las peticiones (POST, PUT...)	Convierte automáticamente el texto del body en un objeto JS (<code>req.body</code>)	Escuchar eventos <code>data</code> y <code>end</code> , concatenar el cuerpo y hacer <code>JSON.parse(body)</code> manualmente
<code>morgan("dev")</code>	Registrar peticiones en consola	Muestra método, URL, estado, tiempo de respuesta, etc.	<code>console.log(req.method, req.url)</code> en cada ruta
<code>cors()</code>	Permitir peticiones desde otros orígenes (dominios diferentes)	Añade cabeceras HTTP <code>Access-Control-Allow-Origin</code> , etc.	Añadir manualmente <code>res.setHeader("Access-Control-Allow-Origin", "*")</code>

En esta sección hemos incorporado tres middlewares fundamentales para cualquier proyecto con Express: `express.json()`, `morgan("dev")` y `cors()`.

El primero, **`express.json()`**, viene incluido dentro de Express, por lo que no es necesario instalar nada adicional. Su función es permitir que el servidor entienda los datos en formato JSON que llegan en el cuerpo de una petición POST, PUT o PATCH. Gracias a este middleware, Express convierte automáticamente ese texto en un objeto JavaScript accesible desde `req.body`, evitando que tengamos que leer los datos manualmente como haríamos en Node puro.

El segundo, **`morgan("dev")`**, sí requiere instalación mediante el comando `npm i morgan`. Este middleware sirve para registrar en consola todas las peticiones que llegan al servidor. Cada vez que un cliente hace una solicitud, morgan muestra información útil como el método (GET, POST, etc.), la ruta, el código de estado y el tiempo de respuesta. Es una herramienta muy práctica para el desarrollo, ya que nos

permite detectar errores y comprobar que las rutas están funcionando correctamente.

Por último, **cors()**, que también se instala con `npm i cors`, permite que otras aplicaciones o dominios externos (como un frontend hecho en React o una web alojada en otro puerto) puedan comunicarse con nuestro servidor sin que el navegador bloquee la conexión. Este middleware añade automáticamente las cabeceras necesarias (Access-Control-Allow-Origin, entre otras) para que la comunicación entre diferentes orígenes sea segura y esté permitida. Ya lo hemos instalado.

En resumen, estos tres middlewares son esenciales en las primeras fases de cualquier proyecto Express:

- `express.json()` facilita recibir datos en formato JSON,
- `morgan("dev")` nos da visibilidad del tráfico en consola,
- y `cors()` garantiza que nuestro servidor sea accesible desde otras aplicaciones.

Con ellos configurados, nuestro servidor queda listo para empezar a trabajar con rutas dinámicas, APIs REST y conexión con frontends modernos.

Vamos a añadir tres middlewares más que también son importantes pues garantizan la seguridad y el buen funcionamiento.

<code>helmet()</code>	<code>npm i helmet</code>	Seguridad HTTP	Añadir cabeceras manualmente
<code>compression()</code>	<code>npm i compression</code>	Comprimir respuestas	Usar <code>zlib</code>
<code>cookie-parser()</code>	<code>npm i cookie-parser</code>	Leer cookies	Analizar <code>req.headers.cookie</code>

Además de los tres middlewares básicos (`express.json()`, `morgan("dev")` y `cors()`), existen otros igualmente importantes que nos ayudan a que el servidor sea más seguro, más eficiente y más completo. Los más recomendables para incorporar a un proyecto Express son `express.static()`, `helmet()` y `compression()`.

El middleware **`express.static()`** viene incluido dentro de Express, por lo que no es necesario instalarlo. Su función es permitir que el servidor sirva archivos estáticos como páginas HTML, hojas de estilo CSS, imágenes o scripts JavaScript. Basta con indicar la carpeta donde se guardan estos archivos, por ejemplo `app.use(express.static("public"))`, y Express se encargará de responder automáticamente cuando el navegador solicite alguno de esos recursos.

En Node puro, esta tarea requeriría leer los archivos manualmente con `fs.readFile()` y configurar las cabeceras adecuadas, algo mucho más laborioso.

Por su parte, **`helmet()`** es un middleware de seguridad que añade automáticamente cabeceras HTTP protectoras frente a ataques comunes como XSS, inyección de código o clickjacking. Para utilizarlo hay que instalarlo desde la terminal con `npm i helmet` e importarlo en el proyecto con `import helmet from "helmet"`; `app.use(helmet());`. Gracias a él, el servidor envía cabeceras como `X-Frame-Options` o `X-Content-Type-Options` sin que tengamos que configurarlas una por una. En un servidor Node sin Express tendríamos que añadir cada cabecera de forma manual, lo que resultaría más propenso a errores.

Por último, **`compression()`** mejora el rendimiento del servidor al comprimir las respuestas antes de enviarlas al cliente. Esto hace que las páginas y los datos viajen más rápido y ocupen menos ancho de banda. Para instalarlo se usa `npm i compression`, y se activa en el código con `app.use(compression());`. Internamente, Express usa el algoritmo `gzip` para reducir el tamaño de los archivos enviados. En Node puro, esta compresión se haría manualmente usando el módulo `zlib`, algo mucho más complejo.

En conjunto, estos tres middlewares —`express.static()`, `helmet()` y `compression()`— convierten a Express en un servidor más completo, seguro y profesional.

`express.static()` facilita servir contenido web, `helmet()` protege las comunicaciones y `compression()` optimiza la velocidad de carga, completando así las herramientas esenciales que todo proyecto Express debería tener desde sus primeras etapas.

En cada proyecto que comiences puedes instalarlos todos a la vez

```
npm i cors morgan helmet compression
```

Middleware	Función
<code>express.json()</code>	Permite leer datos JSON del cliente
<code>morgan("dev")</code>	Registra actividad del servidor
<code>cors()</code>	Permite comunicación entre dominios
<code>express.static()</code>	Sirve archivos web estáticos
<code>helmet()</code>	Aumenta la seguridad
<code>compression()</code>	Acelera la respuesta con compresión

Ejercicios.

Ejercicio 1 – “¡Hola Express!”

Objetivo

Crear un servidor básico con Express, definir las primeras rutas y entender la diferencia entre `res.send()` y `res.json()`.

- Crea un proyecto nuevo con `npm init -y`. Añade `nodemon`.
- Instala Express (`npm i express`) y crea [servidor.js](#). Añade en `.json` las modificaciones que vimos.

- Crea una ruta principal / que devuelva "Bienvenido al servidor Express".
- Añade otra ruta /info que devuelva un objeto JSON con tus datos (nombre, curso, etc.).
- Prueba ambas rutas en el navegador y observa la diferencia entre `res.send()` y `res.json()`.

Paso 1. Arrancar un proyecto express. Modificar el package.json

- `npm init -y` (instala paquete.json)
- `npm i express` (instala express)
- `npm install -D nodemon` (instala nodemon para el modo desarrollo)
- modifica los scripts `"scripts": {`
`"start": "node servidor.js",`
`"dev": "nodemon servidor.js"`
`}`,
 y añade `"type": "module"`.

Paso 2.

Crear el archivo [servidor.js](#)

Ponle una respuesta para la ruta general y otra para la ruta /info.

Método	Qué hace	Tipo de contenido	Uso típico
<code>res.send()</code>	Envía texto o HTML al cliente y cierra la respuesta.	<code>text/html</code> o <code>text/plain</code>	Mensajes, vistas, HTML simples
<code>res.json()</code>	Envía un objeto JavaScript convertido a JSON.	<code>application/json</code>	APIs o respuestas de datos

Ejercicio 2 – “Mi primera respuesta HTML”

Partimos del proyecto del ejercicio 1

Tienes express instalado y un `servidor.js` con la app creada.

Añade una ruta que devuelva HTML

En Express puedes enviar HTML directamente con `res.send()`. Basta con pasarle un string con etiquetas. Prueba tal cual, por ejemplo en la ruta contacto ponlo para que el usuario que visite esta ruta vea:

```
<h3>Esta es la página de Contacto.</h3>"
+ "<p>Puedes contactarnos a través de <a
href='mailto:contacto@ejemplo.com'>contacto@ejemplo.com</a>.</p>"
```

El método `res.send()` es la forma más sencilla de responder a una petición en Express.

Sirve para enviar texto plano, HTML o incluso JSON, y el navegador mostrará el contenido según el tipo de datos enviado.

Ahora vamos a poner también un mensaje de error para aquellas rutas que no existan. Usa `app.use` y `res.status`.

`app.use()` es un método de Express que sirve para registrar middlewares o funciones intermedias que se ejecutan cada vez que llega una petición al servidor.

Tanto `app.get()` como `app.use()` definen funciones que se ejecutan cuando llega una petición, pero no hacen exactamente lo mismo.

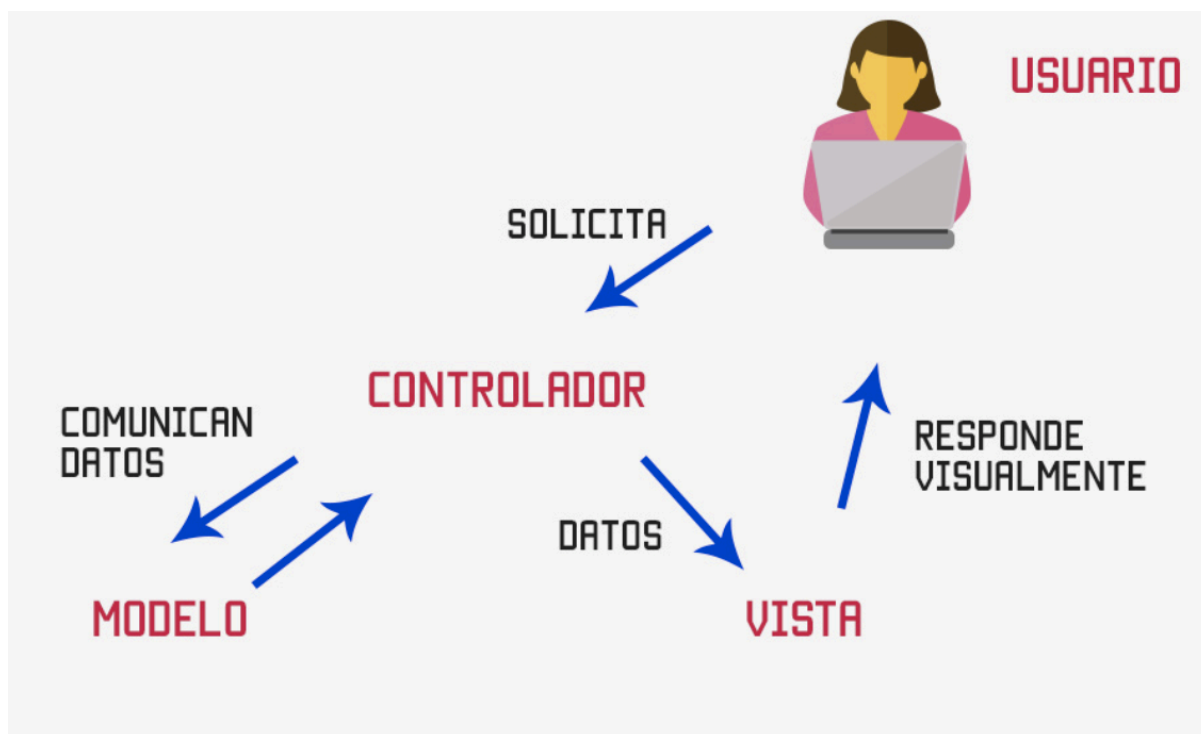
<code>app.get(ruta, callback)</code>	Crea una ruta específica que responde a peticiones GET	Solo cuando el cliente hace una petición GET a esa ruta concreta	Para páginas o endpoints concretos (<code>/</code> , <code>/info</code> , <code>/contacto</code> , etc.)
<code>app.use([ruta], callback)</code>	Registra un middleware que se ejecuta para cualquier método HTTP (GET, POST, PUT, DELETE...), en todas las rutas o en las que empiecen por la indicada	Siempre que pase una petición por esa ruta (o por todas si no se indica ninguna)	Para middlewares globales (CORS, logs, 404, validaciones, seguridad, etc.)

Para el cierre usamos `app.use` pues no tiene ruta, por lo que se ejecuta si ninguna ruta anterior (GET, POST, etc.) coincide. Por eso lo usamos para el 404, el “cajón de cierre” del servidor.

5.Introducción al Modelo MVC (Modelo–Vista–Controlador)

Ahora que nuestras aplicaciones comienzan a crecer es importante establecer una estructura que nos permita escalar de una forma ordenada.

El MVC o Modelo-Vista-Controlador es un patrón de arquitectura de software que, utilizando 3 componentes (Vistas, Models y Controladores) separa la lógica de la aplicación de la lógica de la vista en una aplicación.



¿Por qué se usa este modelo?

Nos permite separar los componentes de nuestra aplicación dependiendo de la responsabilidad que tienen, esto significa que cuando hacemos un cambio en alguna parte de nuestro código, esto no afecta a otra parte del mismo. Por ejemplo, si modificamos nuestra Base de Datos, sólo deberíamos modificar el modelo que es quién se encarga de los datos y el resto de la aplicación debería permanecer intacta. Esto respeta el principio de la responsabilidad única. Es decir, una parte de tu código no debe de saber qué es lo que hace toda la aplicación, sólo debe de tener una responsabilidad.

En web, el MVC funciona así. Cuando el usuario manda una petición al navegador, digamos quiere ver el curso de ReactJS, el controlador responde a la solicitud, porque él es el que controla la lógica de la app, una vez que el controlador nota que el usuario solicitó el curso de React, le pide al modelo la información del curso. El modelo, que se encarga de los datos de la app, consulta la base de datos y digamos, obtiene todos los vídeos del curso de ReactJS, la información del curso y el título, el modelo responde al controlador con los datos que pidió (nota como en la imagen las flechas van en ambos sentidos, porque el controlador pide datos, y el modelo responde con los datos solicitados).

Una vez el controlador tiene los datos del curso de ReactJS, se los manda a la vista, la vista aplica los estilos, organiza la información y construye la página que ves en el navegador.

Modelo

Se encarga de los datos, generalmente (pero no obligatoriamente) consultando la base de datos. Actualizaciones, consultas, búsquedas, etc. todo eso va aquí, en el modelo. Las validaciones también se harían en el modelo.

Controlador

Se encarga de controlar, recibe las órdenes del usuario y se encarga de solicitar los datos al modelo y de comunicarles a la vista.

Vistas

Son la representación visual de los datos, todo lo que tenga que ver con la interfaz gráfica va aquí. Ni el modelo ni el controlador se preocupan de cómo se verán los datos, esa responsabilidad es únicamente de la vista.

El patrón MVC (Modelo–Vista–Controlador) ha sido históricamente uno de los pilares del desarrollo web, especialmente en frameworks que integraban las tres capas dentro del mismo entorno (como Laravel, Rails o Django).

Sin embargo, la evolución de la web hacia aplicaciones más interactivas, con interfaces dinámicas ejecutadas en el navegador, ha cambiado el equilibrio de responsabilidades entre frontend y backend.

En las arquitecturas modernas, el frontend ya no depende del servidor para renderizar HTML. En su lugar, utiliza frameworks como React, que gestionan la interfaz a través de componentes, páginas y estado.

Por ello, el patrón MVC completo pierde sentido en el lado del servidor, donde ya no necesitamos una “Vista” como tal.

En este proyecto, decidimos mantener la estructura del Modelo y el Controlador en el backend (para gestionar datos, lógica y API REST), y trasladar la Vista al frontend, que se encarga de renderizar la información, controlar la interacción del usuario y actualizar la interfaz de forma reactiva.

Esta división nos permite:

- Diseñar un backend limpio, centrado en lógica y persistencia de datos.
- Desarrollar un frontend autónomo, reactivo y reutilizable.
- Favorecer la escalabilidad y la independencia tecnológica (podríamos cambiar el frontend sin tocar la API).
- Adoptar un enfoque full stack moderno, donde cliente y servidor cooperan mediante fetch y respuestas JSON.

En resumen, intentamos combinar lo mejor de ambos mundos:

la claridad estructural de MVC en el backend y la flexibilidad de los componentes en el frontend. Esta decisión refleja el modo en que hoy trabajan los equipos profesionales en entornos de desarrollo web modernos.

Ejemplo cotidiano para entenderlo

Imagina una tienda online:

- El Modelo sería el almacén de productos: sus precios, stock, categorías...
- El Controlador sería el dependiente: recibe la petición del cliente (“quiero ver zapatillas”) y decide qué datos mostrar.

- La Vista sería el escaparate o la web: enseña la información de forma visual.

¿Y esto cómo lo llevamos a la práctica?

6.API REST con Express (CRUD básico)

Estructura.

¿Qué vamos a construir?

- Un servidor (backend) con Express que expone una API REST para gestionar libros (listar, crear, editar, borrar).
- Un cliente (frontend) con HTML/CSS/JS que consume esa API y muestra una tabla de libros.

Filosofía: el backend se encarga de los datos y la lógica; el frontend se encarga de la interfaz.

Usaremos Modelo + Controlador en el backend, y una página con JS en el frontend. En el frontend vamos a probar la librería Axios para facilitar las conexiones con HTTP.

Cómo encaja Express dentro del modelo MVC

Express se utiliza para construir el lado del servidor (backend), así que encaja principalmente en la capa Controlador, aunque también puede facilitar el acceso al Modelo.

Distribución típica en un proyecto Node + Express:

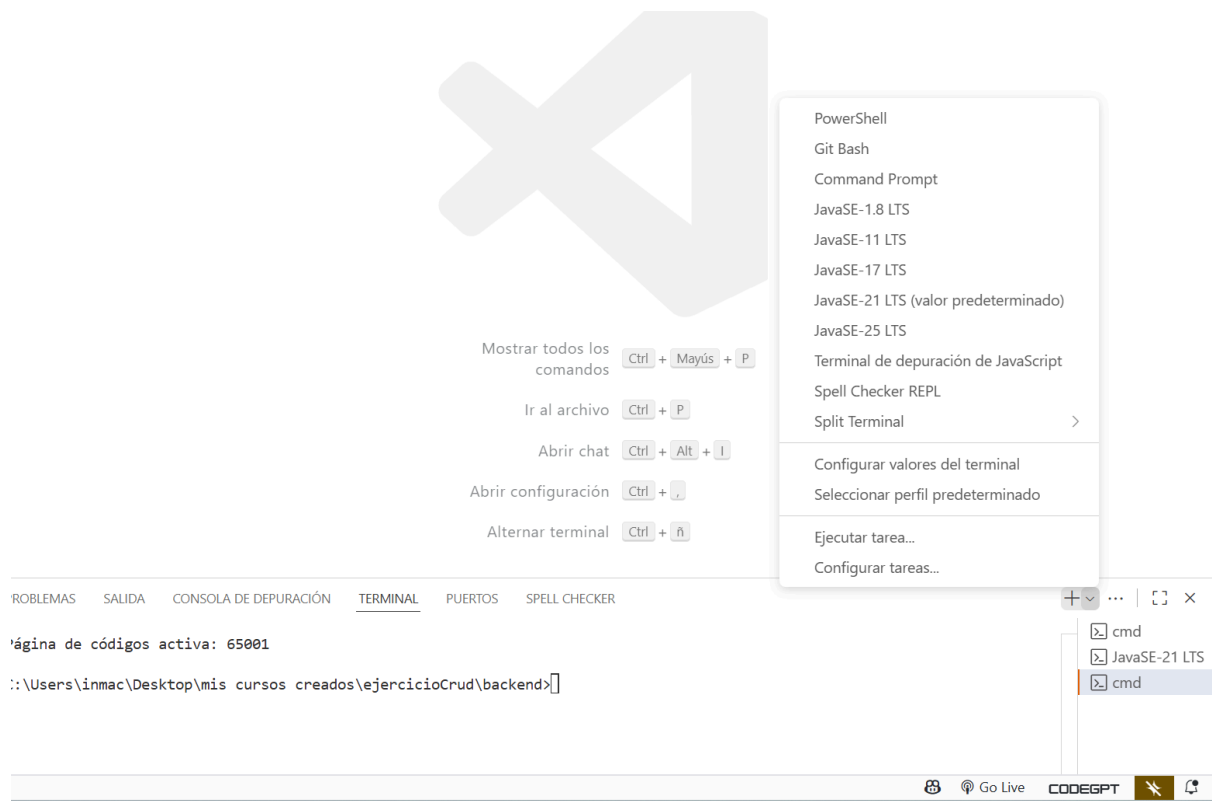
Express actúa como “pegamento” entre el cliente y el modelo.

Cuando llega una petición:

1. Express detecta la ruta (por ejemplo /api/productos).
2. Llama al controlador correspondiente.
3. El controlador pide los datos al modelo.
4. El modelo devuelve los datos al controlador.
5. El controlador responde al cliente (por ejemplo, con `res.json()`).

Vamos a crear esta estructura. Te dejo un atajo: creamos una carpeta con nuestro proyecto con el nombre que quieras. Dentro dos carpetas una para backend y otra para frontend.

Nos metemos dentro de backend y abrimos la carpeta backend en la terminal
Dentro de esta carpeta ejecutamos en el power shell abrelo en la flecha de la derecha



Asegúrate que estás dentro de la carpeta backend si no estás dentro te mueves con `cd backend`.

En el powershell ejecutamos

`mkdir models, controllers, routes, data`

`ni models/libros.model.js, controllers/libros.controller.js, routes/libros.routes.js,`

`data/libros.json, server.js -ItemType File`

¿Qué es el power shell?

PowerShell es una terminal (una línea de comandos) que viene con Windows, pero mucho más moderna y potente que la antigua CMD (el "Símbolo del sistema").

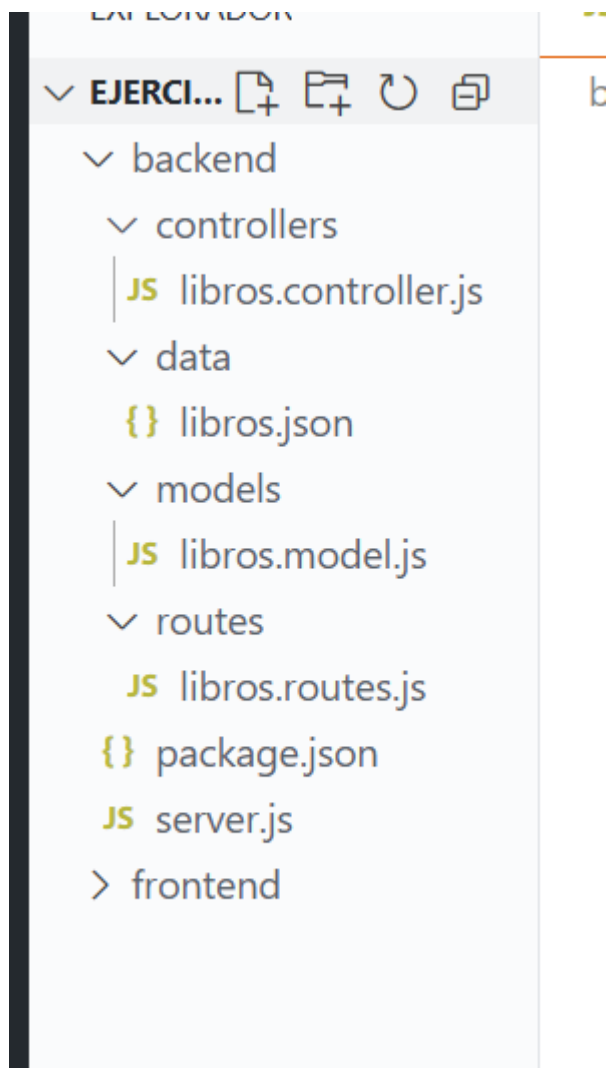
Piensa que es como una "versión avanzada" de CMD:

te permite ejecutar comandos, crear carpetas, automatizar tareas y hasta programar scripts con su propio lenguaje.

PowerShell tiene su propio lenguaje de scripting, llamado: PowerShell Script

Language (basado en .NET). Nos puede recordar a Linux y algunos comandos se parecen pero presenta diferencias.

Ahora tenemos algo parecido a esto



Paso 1. Preparamos nuestro entorno de desarrollo.

¿Con qué vamos a trabajar? En nuestro caso vamos a trabajar con Node y con Express en el backend.

Nos vamos a la carpeta backend y la abrimos en el terminal.

Ejecutamos:

```
npm init -y
```

Como ya sabemos esto nos crea nuestro archivo package.json. Abre package.json y añade la línea "type": "module". Así podremos usar import y export (en lugar de require).

Ahora vamos a instalar express con sus middlewares básicos siempre dentro de la carpeta del back

npm i express cors morgan helmet compression

Ahora vamos a instalar nodemon para poder ejecutar con npm run dev

npm i -D nodemon

Lo escribimos en los scripts de nuestro package.json; este no se escribe solo.

Dejamos el de start y añadimos

```
"dev": "nodemon server.js",
```

quedará así

```

  ▶ Depurar
  "scripts": {
    "dev": "nodemon server.js",
    "start": "node server.js"
  },
```

Ahora ya tenemos el entorno preparado para empezar a trabajar.

Paso 2. Vamos a crear nuestra pequeña bbdd con datos json. Con ella lograremos la persistencia es decir nuestros datos no se borrarán al salir del navegador.

Estamos en la carpeta data en el fichero libros.json

Te lo puedes crear vacío [] o bien meterle algún libro

```
[
  { "id": 1, "titulo": "Donde los árboles cantan", "autor": "Laura Gallego", "anio": 2011 },
  { "id": 2, "titulo": "La chica del tren", "autor": "Paula Hawkins", "anio": 2015 },
  { "id": 3, "titulo": "Patria", "autor": "Fernando Aramburu", "anio": 2016 },
  { "id": 4, "titulo": "El tatuador de Auschwitz", "autor": "Heather Morris", "anio": 2018 },
```

```
{ "id": 5, "titulo": "Reina Roja", "autor": "Juan Gómez-Jurado", "anio": 2018 }  
]
```

Paso 2. Vamos a crear nuestra capa de datos. Es decir la que va a acceder a este fichero de datos json. Este es el fichero que sabe cómo acceder a los datos da igual si estos están en un bbdd o un archivo json. Es el único archivo que accede a los datos, el controlador pide al modelo que le envíe los datos. Este archivo es puro javascript no maneja rutas sólo accede a los datos y los prepara para enviarlos al controlador.

Modelo.

```
// models/libros.model.js  
//  Modelo: se encarga de leer y escribir el archivo JSON donde guardamos los libros.
```

```
import { readFile, writeFile } from "fs/promises"; // versión moderna (usa promesas)
```

Es como ya sabemos un modulo nativo y es el que debemos usar si encadenamos varias acciones sobre archivos devuelve promesas por eso await.

```
import path from "path";  
import { fileURLToPath } from "url";  
  
// Obtenemos la ruta absoluta del archivo libros.json  
const __dirname = path.dirname(fileURLToPath(import.meta.url));  
const RUTA_ARCHIVO = path.join(__dirname, "..", "data", "libros.json");
```

```
/**
```

Vamos a analizar algunas de las líneas que pueden resultar más curiosas o difíciles de entender.

La creación de rutas. Usamos el módulo nativo `path`. En los proyectos de Node con "type": "module", las variables especiales `__dirname` y `__filename` ya no existen por defecto pero existirán en la próxima versión. De momento, , debemos reconstruir la ruta del archivo actual con ayuda de dos módulos nativos de Node:

- `fileURLToPath(import.meta.url)` convierte la dirección interna del archivo (una URL) en una ruta normal del sistema, por ejemplo:
file:///C:/Users/Inma/Desktop/backend/models/libros.model.js →
C:\Users\Inma\Desktop\backend\models\libros.model.js
- `path.dirname(...)` toma esa ruta completa y se queda solo con la carpeta donde está el archivo actual (models en este caso).

Con eso, guardamos en la constante `__dirname` la ruta exacta del directorio actual, sin importar si estamos en Windows, Linux o Mac.

Finalmente, `path.join(__dirname, "..", "data", "libros.json")` sube una carpeta (..) y entra en data/libros.json, construyendo la ruta absoluta hacia nuestro archivo JSON de datos.

```
* 📖 Lee la lista de libros del archivo JSON.  
* Si el archivo no existe, devuelve un array vacío.  
*/  
export async function leerLibros() {  
  try {  
    // Leemos el archivo completo como texto  
    const texto = await readFile(RUTA_ARCHIVO, { encoding: "utf8" });
```

Aquí lo que está leyendo es texto porque un archivo .json es texto aunque parezcan objetos por eso hay que convertirlo en objetos para integrarlo en un array y poder trabajar con ellos. Cuando lo leemos con `readFile`, obtenemos una cadena de texto. Con `JSON.parse(texto)` lo transformamos en un array de objetos JavaScript que el resto de la app puede manipular:

```
// Lo convertimos de texto JSON → objeto JS (array de libros)
```

```
return JSON.parse(texto);
```

Ahora sí, `datos` es un array real que puedes recorrer, filtrar o modificar. Cuando usamos una base de datos, no necesitamos `parse`, porque el driver (por ejemplo `mysql2/promise`) ya devuelve los registros como objetos JS.

```
} catch (error) {  
  // ENOENT = archivo no encontrado → devolvemos lista vacía  
  if (error.code === "ENOENT") {  
    return [];  
  }  
  // Si el error es otro (por ejemplo JSON roto), lo re-lanzamos  
  throw error;  
}
```

¿Qué hace esta captura de error? Paso a paso:

1. `catch (err)`: captura cualquier error que ocurra dentro del `try`, por ejemplo:

- el archivo `libros.json` no existe,
- el archivo está dañado o no tiene formato JSON válido,
- o hay un problema de permisos en disco.

2. `if (err.code === "ENOENT")`:

Esta línea detecta específicamente el error "archivo no encontrado".

- `ENOENT` significa: Error NO ENTry (no existe la entrada o ruta).
`ENOENT` es el estado que envía el módulo `fs`. Por eso lo captura el error como `ENOENT`.

Cuando intentas leer un archivo que no existe, el sistema lanza un error con esta estructura:

```

{
  errno: -4058,
  code: "ENOENT",
  syscall: "open",
  path: "C:\\ruta\\que\\no\\existe.json"
}

```

- Es un código estándar de error del sistema de archivos de Node.js.

3. En lenguaje humano: “No encuentro el archivo libros.json”.

4. Qué hace si pasa eso:

Si el archivo no existe (primera vez que ejecutas el servidor),
entonces lo crea vacío (`await guardarLibros([])`)
y devuelve una lista vacía `[]` para que la app siga funcionando.

5. `throw err;`

Si el error no es `ENOENT` (por ejemplo, el JSON está roto o no hay permisos),
entonces lo vuelve a lanzar (re-lanza) para que el middleware de errores de
Express lo capture y muestre un mensaje apropiado.

*/*permite guardar la lista de libros en el archivo JSON; y a la vez permite usar esa
función por otros archivos*/*

```

export async function guardarLibros(listaDeLibros) {
  // JSON.stringify convierte el array en texto bonito con sangría de 2 espacios

```

Aquí hacemos lo contrario convierte un objeto o array JS a texto JSON para poder guardarlo en disco. `JSON.stringify` convierte el array en texto JSON.

Los argumentos null, 2 sirven para que quede formateado con sangría de 2 espacios, legible por humanos. Luego writeFile escribe ese texto dentro de libros.json.

```
const texto = JSON.stringify(listaDeLibros, null, 2);
// Escribimos el archivo con el contenido actualizado
await writeFile(RUTA_ARCHIVO, texto, { encoding: "utf8" });
}

export function nuevold(lista) {
  const max = lista.reduce((m, l) => Math.max(m, Number(l.id) || 0), 0);
  return max + 1;
}
```

Esta función lo que hace es que recorre el array para ver cual es el número más alto que existe en el array y le asigna el siguiente al libro creado.

Acuérdate del esquema nuestra capa de modelo no se comunica con el navegador va a ser el controlador el que le va a pedir los datos. Por eso en esta capa no usa Express. Express sirve para manejar: rutas (URL), peticiones (req), respuestas (res) y middlewares (autenticación, validación, CORS, etc.). Y nada de eso ocurre dentro del modelo.

En conclusión, este archivo es el modelo (M) de nuestra aplicación MVC.

Su misión es gestionar los datos de los libros, accediendo al archivo data/libros.json para:

- Leer la lista de libros
- Guardar los cambios (crear, editar, borrar)

- Generar un nuevo id automático

El modelo no decide qué se borra o actualiza, sino cómo acceder y guardar los datos.

```
CLIENTE (frontend)
  ↓ petición GET /api/productos
CONTROLADOR (productos.controller.js)
  ↓ llama al modelo
MODELO (productos.model.js)
  ↓ lee datos del archivo JSON
CONTROLADOR
  ↓ envía res.json(datos)
EXPRESS
  ↓
CLIENTE ← muestra los datos en la vista
```

Controlador.

1) ¿Qué es next y para qué lo usamos?

- next es el tercer parámetro del controlador (después de req y res) que te da Express.
- Si lo llamas con un error: next(error), Express salta al middleware de manejo de errores (el que tienes al final de server.js con 4 parámetros: (error, req, res, next)).
- Ventaja: centralizas cómo respondes a errores (p.ej., siempre con JSON { error: ... } y status 500).

Ejemplo típico en server.js:

```
app.use((error, req, res, next) => {  
  console.error("💥 Error:", error);  
  res.status(500).json({ error: "Error interno del servidor" });  
});
```

2) catch (error) – ¿qué está capturando?

- Captura cualquier fallo que ocurra en el try, por ejemplo:
 - Lectura/escritura del archivo JSON,
 - Propiedad que no existe,
 - Conversión de tipos, etc.
- En nuestro código, no respondemos directamente en el catch, sino que hacemos `next(error)` para que el middleware de errores se encargue.

3) ¿Qué significa esta línea y por qué la “llave vacía”?

```
const { titulo, autor, anio } = req.body || {};
```

- Desestructuración: saca titulo, autor, anio de req.body.
- `|| {}` (objeto vacío como respaldo): si por alguna razón req.body es undefined (p. ej., si no pusiste `app.use(express.json())`), no fallará la desestructuración. Es decir, evita un error del tipo: “Cannot destructure property 'titulo' of 'req.body' as it is undefined”.

Luego validamos lo obligatorio:

```
if (!titulo || !autor) {  
  return res.status(400).json({ error: "Faltan campos obligatorios: titulo y autor" });  
}
```

-

4) ¿Qué hace exactamente trim()?

- String(valor).trim() convierte a string y elimina espacios al principio y al final.
- Útil para no guardar " Don Quijote " con espacios extra.
- Ejemplos:
 - " hola ".trim() → "hola"
 - String(123).trim() → "123"

5) ¿Cómo funciona actualizar paso a paso?

1. Leemos id de la URL y posibles campos del body.
2. Leemos todos los libros y buscamos el índice del libro con ese id.
3. Si no existe, 404.
4. Creamos un objeto actual a partir del existente y solo cambiamos los campos que vengan en el body (titulo, autor, anio).
 - Esto permite una actualización parcial (si solo mando autor, no toco titulo ni anio).
5. Sustituimos en el array y guardamos la lista.
6. Respondemos con el libro actualizado.

- Controlador = “intérprete entre HTTP y los datos”.
- req → lo que llega (params, body).
- res → lo que devuelves (JSON + status).
- next(error) → delega el error al manejador global.
- Desestructuración con `|| {}` → evita petadas si req.body no existe.
- trim() → limpia espacios.
- Actualizar parcial (PUT) → solo cambias lo que te envían.

Axios. En el Frontend

Sustituyendo fetch() por Axios

Al analizar el código del frontend, habrás notado que ya no aparece fetch() por ningún lado.

Esto se debe a que lo hemos sustituido por una librería externa llamada Axios, que simplifica y mejora las conexiones HTTP entre el frontend (cliente) y el backend (servidor Express).

¿Qué es Axios?

Axios es una librería JavaScript (de terceros) que se utiliza para hacer peticiones HTTP (GET, POST, PUT, DELETE...) de manera más sencilla y potente que fetch().

Se instala fácilmente mediante una URL o con NPM:

<!-- Para usarlo directamente desde el navegador -->

<script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>

O, si trabajas con Node.js o React:

`npm install axios`

Una vez instalada, podemos usarla igual que `fetch()`, pero con más control y menos código. Ambas herramientas sirven para lo mismo: Comunicar tu frontend (HTML/JS) con un backend (por ejemplo, Express o una API externa) usando peticiones HTTP. Pero se comportan de manera diferente al enviar o recibir datos JSON.

Antes: usando `fetch()`

Cuando no existía Axios, utilizábamos el método nativo del navegador:

```
fetch("http://localhost:3000/api/libros")
  .then(res => res.json())
  .then(data => console.log("Libros:", data))
  .catch(error => console.error("Error:", error));
```

Problemas de `fetch()`:

- ❌ No detecta bien los errores del servidor (por ejemplo, errores 404 o 500).
- ❌ Hay que convertir manualmente la respuesta con `.json()`.
- ❌ No permite configurar cabeceras globales fácilmente.
- ❌ No tiene interceptores ni soporte cómodo para autenticación o tokens.

Ahora: usando Axios

Con Axios, el mismo código se simplifica enormemente:

```
const { data } = await axios.get("http://localhost:3000/api/libros");
console.log("Libros:", data);
```

Y para crear, actualizar o eliminar recursos:

```
await axios.post(API, nuevoLibro);    // Crear
await axios.put(`${API}/${id}`, cambios); // Actualizar
await axios.delete(`${API}/${id}`);    // Eliminar
```

Código para crear un libro con Fetch.

```
fetch("http://localhost:3000/api/libros", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    titulo: "1984",
    autor: "George Orwell"
  })
})
.then(res => res.json()) // 👉 hay que parsear manualmente
.then(data => console.log("✅ Libro creado:", data))
.catch(err => console.error("❌ Error:", err));
```

Código para crear un libro con Axios.

```
await axios.post("http://localhost:3000/api/libros", {
  titulo: "1984",
  autor: "George Orwell"
});
```

Axios ya hace automáticamente el `JSON.stringify()` al enviar y el `JSON.parse()` al recibir, así que tu código queda más legible y seguro. Axios automatiza esos dos pasos:

- Si le pasas un objeto, lo convierte a JSON internamente.

- Si el servidor responde con JSON, lo transforma automáticamente a objeto JS en `response.data`.

En otras palabras:

Con Axios ya no hablamos “en texto”, hablamos directamente en objetos JavaScript.

Cuando usas `fetch()`, estás trabajando con texto plano (strings),

así que necesitas convertirlo manualmente a objeto JS o viceversa: